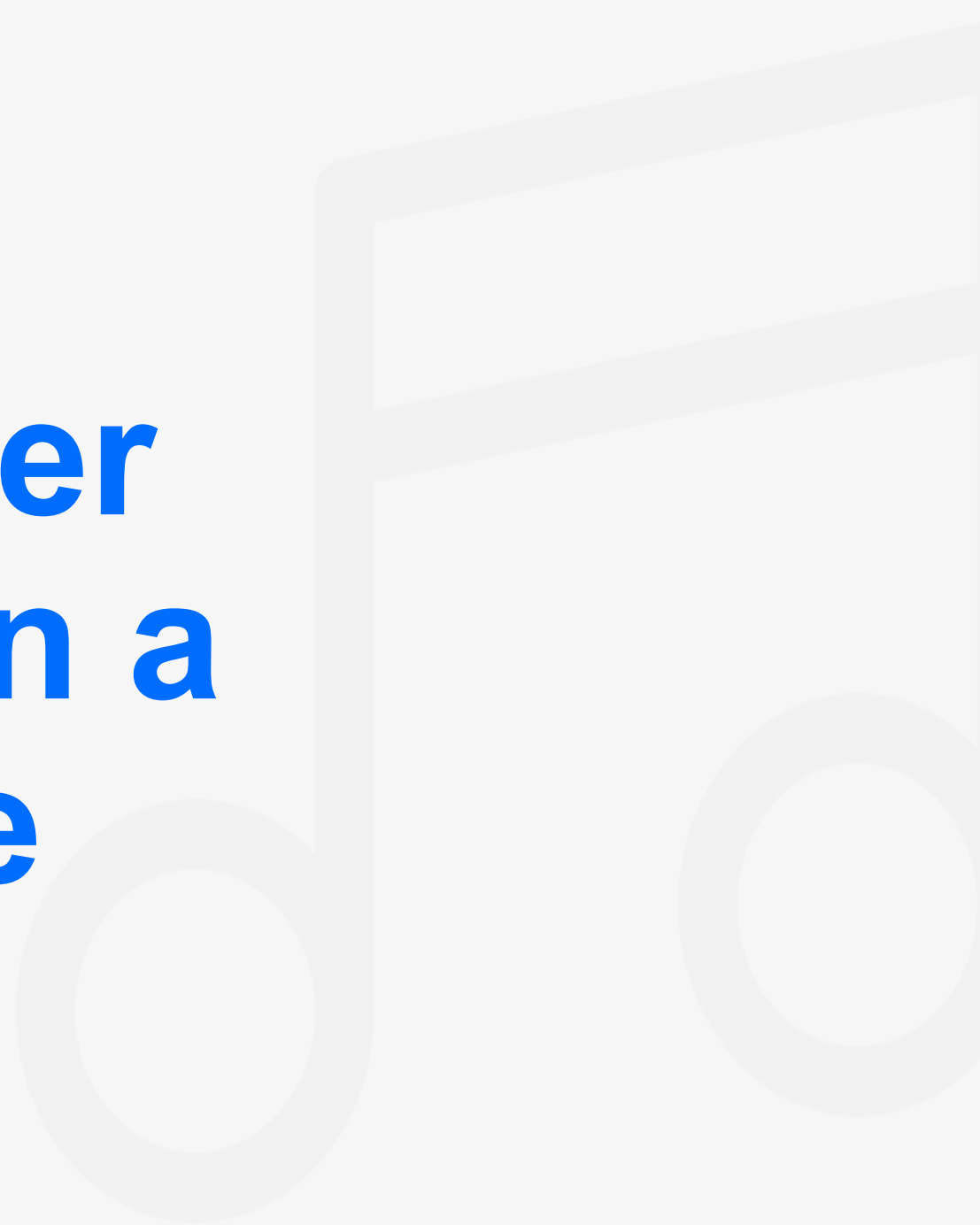


Predicting user preferences in a music service



Dataset

Dataset

Source: Kaggle huynguyen1902/music-interaction

Table	Rows	Size	Key columns
tracks_features	1.2 M	330 MB	id, danceability, energy, valence, tempo, loudness...
interaction	11.8 M	574 MB	user_id, item_id, interaction_flag, ts (Unix timestamp)

Target: interaction_flag (1 = positive engagement)

Raw timestamp not used directly, decomposed into cyclical features later.

Pipeline flow

Pipeline flow

Raw CSV → PostgreSQL (COPY) → Sqoop → HDFS (AVRO+Snappy)



Hive AVRO external → ORC partitioned/bucketed tables



Spark on YARN reads Hive → Feature Engineering → ML Training

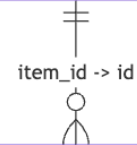


Model outputs → Hive external tables → Apache Superset

Stage I

PostgreSQL Schema

TRACKS		
VARCHAR	id	PK
TEXT	name	
TEXT	artists	
BOOLEAN	explicit	
DOUBLE	danceability	
DOUBLE	energy	
DOUBLE	valence	
DOUBLE	tempo	
INTEGER	year	



INTERACTIONS		
SERIAL	id	PK
VARCHAR	user_id	
VARCHAR	item_id	FK
INTEGER	interaction_flag	
BIGINT	ts	

tracks: id PK, name, artists, danceability, energy, ... (23 columns)

interactions: id PK, user_id, item_id FK→ tracks.id, interaction_flag, ts BIGINT
Indexes on interactions.user_id, item_id, ts for join and time-based queries.

Loading method: COPY ... FROM STDIN via Python psycopg2, 11.8 M rows

Stage I

Sqoop Import & HDFS Layout

Sqoop command executed on hadoop-01:

```
bash
sqoop import-all-tables \
  --connect jdbc:postgresql://hadoop-04/team11_projectdb \
  --username team11 --password-file ... \
  --compression-codec=snappy --compress \
  --as-avrodatafile \
  --warehouse-dir=project/warehouse --m 1
```

Output in
HDFS: project/warehouse/tracks/ and project/warehouse/interactions/
AVRO files compressed with Snappy (~30% size reduction)
Generated .avsc and .java artifacts stored in output/

Why AVRO+Snappy? AVRO embeds schema, perfect for Sqoop→Hive handoff; Snappy offers fast compression suitable for repeated development runs.

Stage II

Hive Warehouse Design

Two-layer table architecture

- Staging tables dropped from metastore after optimized tables are materialized.
- Hive settings: `execution.engine=tez`, dynamic partitioning, `orc.compress=SNAPPY`.

Layer	Table	Format	Partition	Buckets
Staging	tracks	AVRO	–	–
Staging	interactions	AVRO	–	–
Optimized	tracks_part	ORC + Snappy	year	id (11)
Optimized	interactions_part	ORC + Snappy	interaction_flag	user_id (11)

Stage II

EDA Overview

Six HiveQL queries:

q1 – Class balance of interaction_flag

q2 – Artist loyalty (positive interactions per user-artist pair)

q3 – Average audio features by class

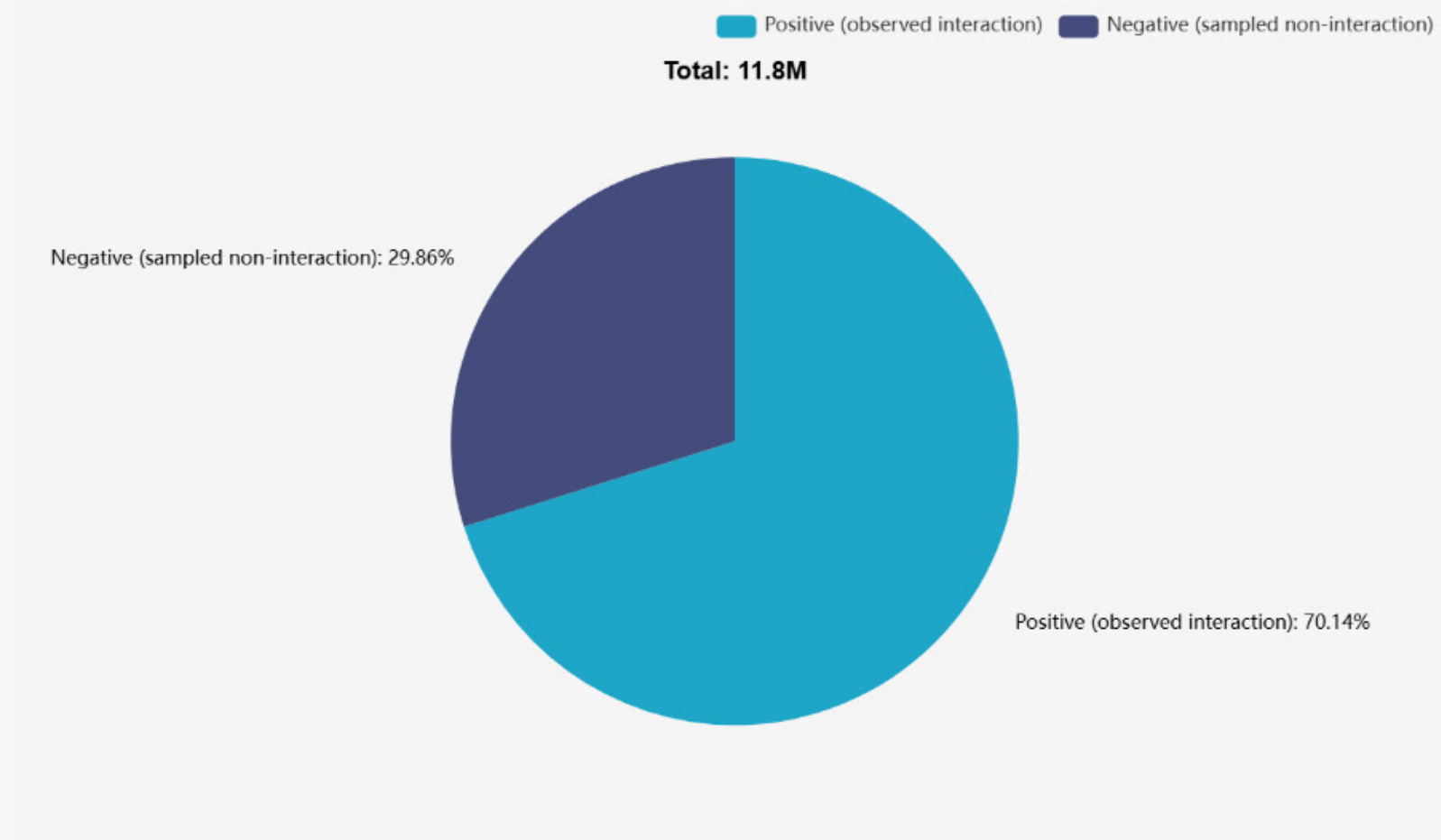
q4 – Positive rate by track popularity bucket

q5 – User activity distribution

q6 – Overall vs cold catalogue vs hit tracks positive rate

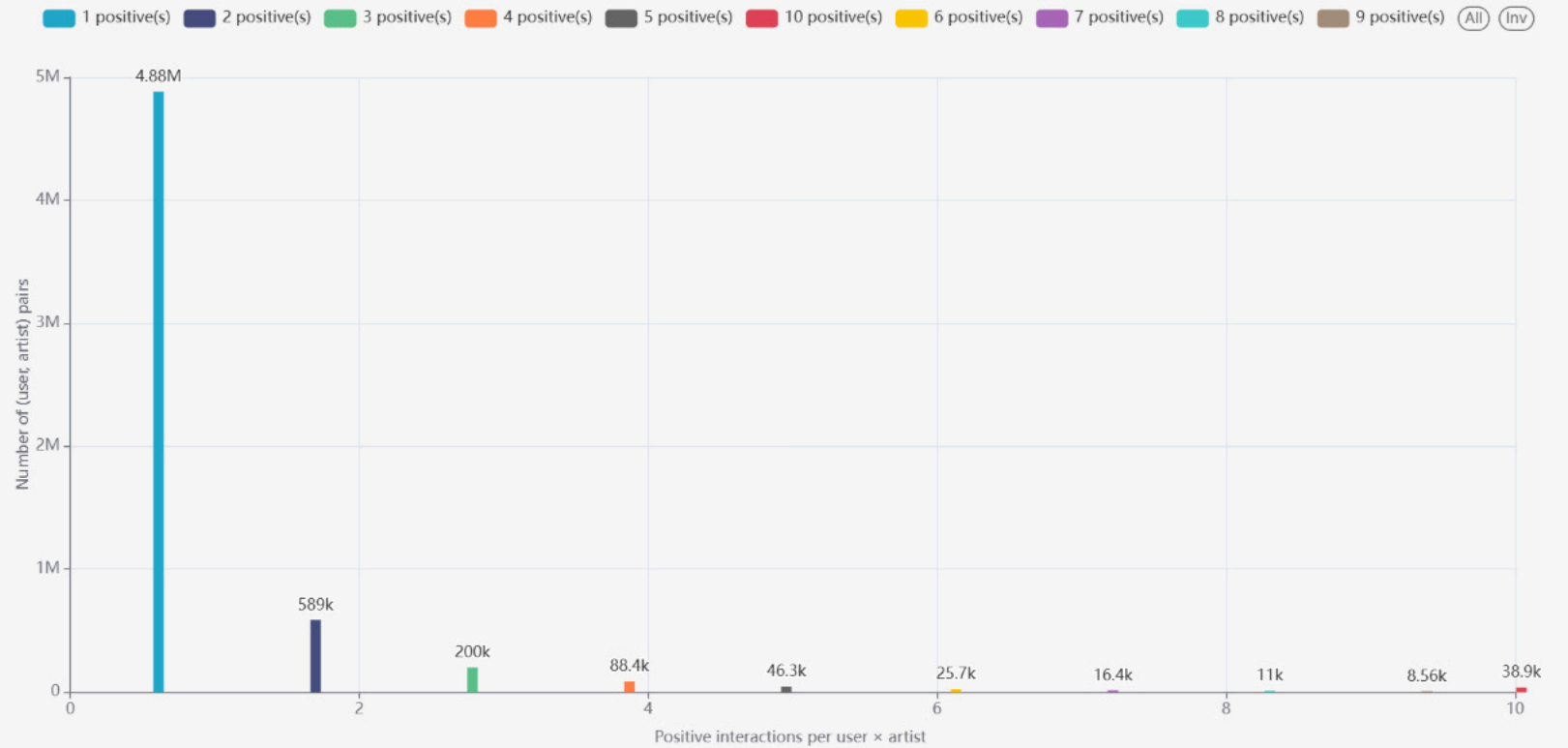
Stage II

Q1 – class balance



Stage II

Q2 – Artist loyalty



Stage II

Q3 – Average audio
features by class

feature	positive	negative
danceability	0.614	0.493
energy	0.643	0.509
valence	0.511	0.428
acoustiness	0.224	0.447
loudness	-7.32	-11.81
tempo	120.95	117.65

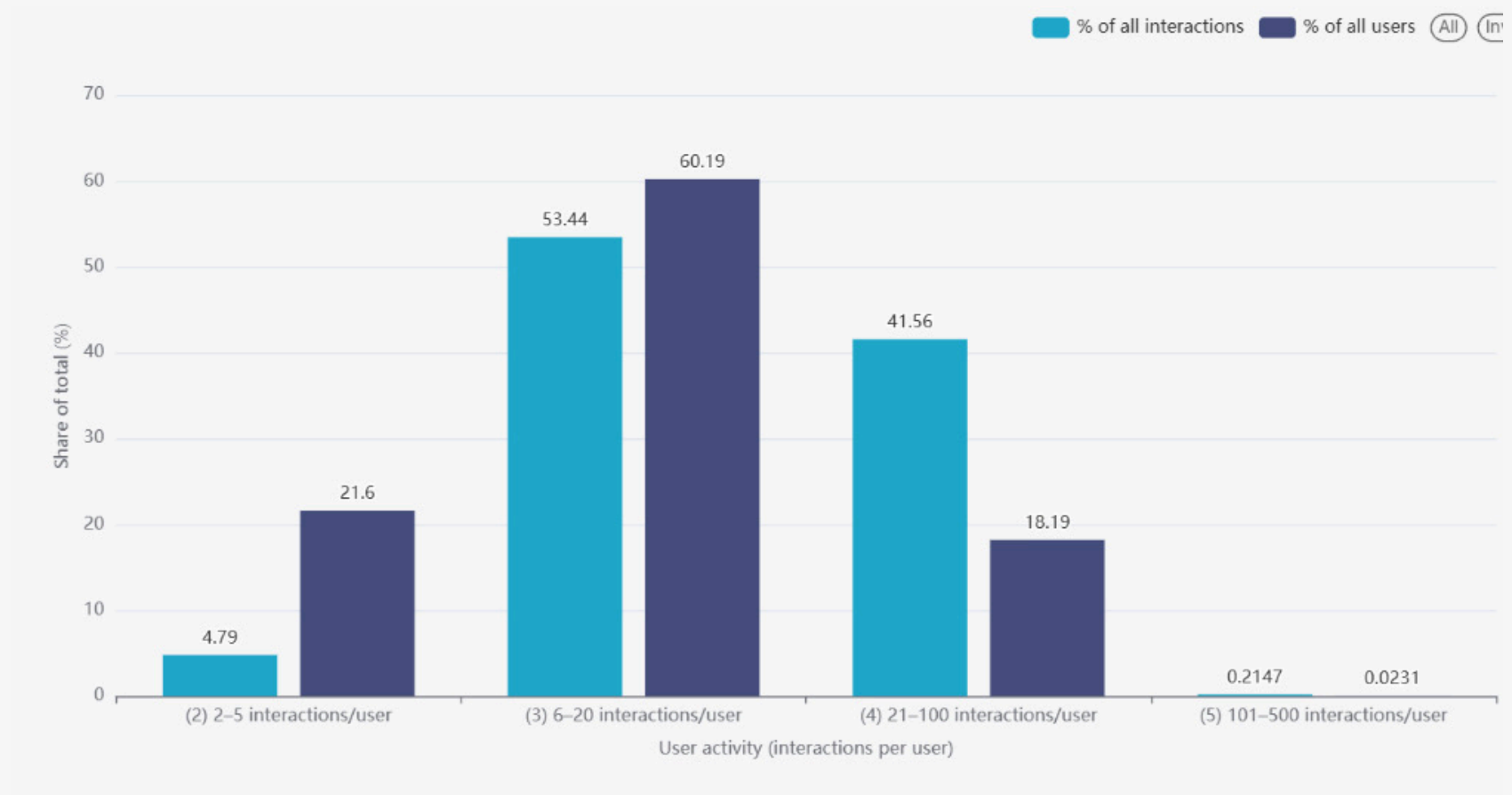
Stage II

Q4 – Positive rate by popularity bucket

bucket	positive rate
1 interaction	0.016
2-3 interactions	0.023
4-10 interactions	0.083
11-30 interactions	0.797
31-100 interactions	0.944
100+ interactions	0.997

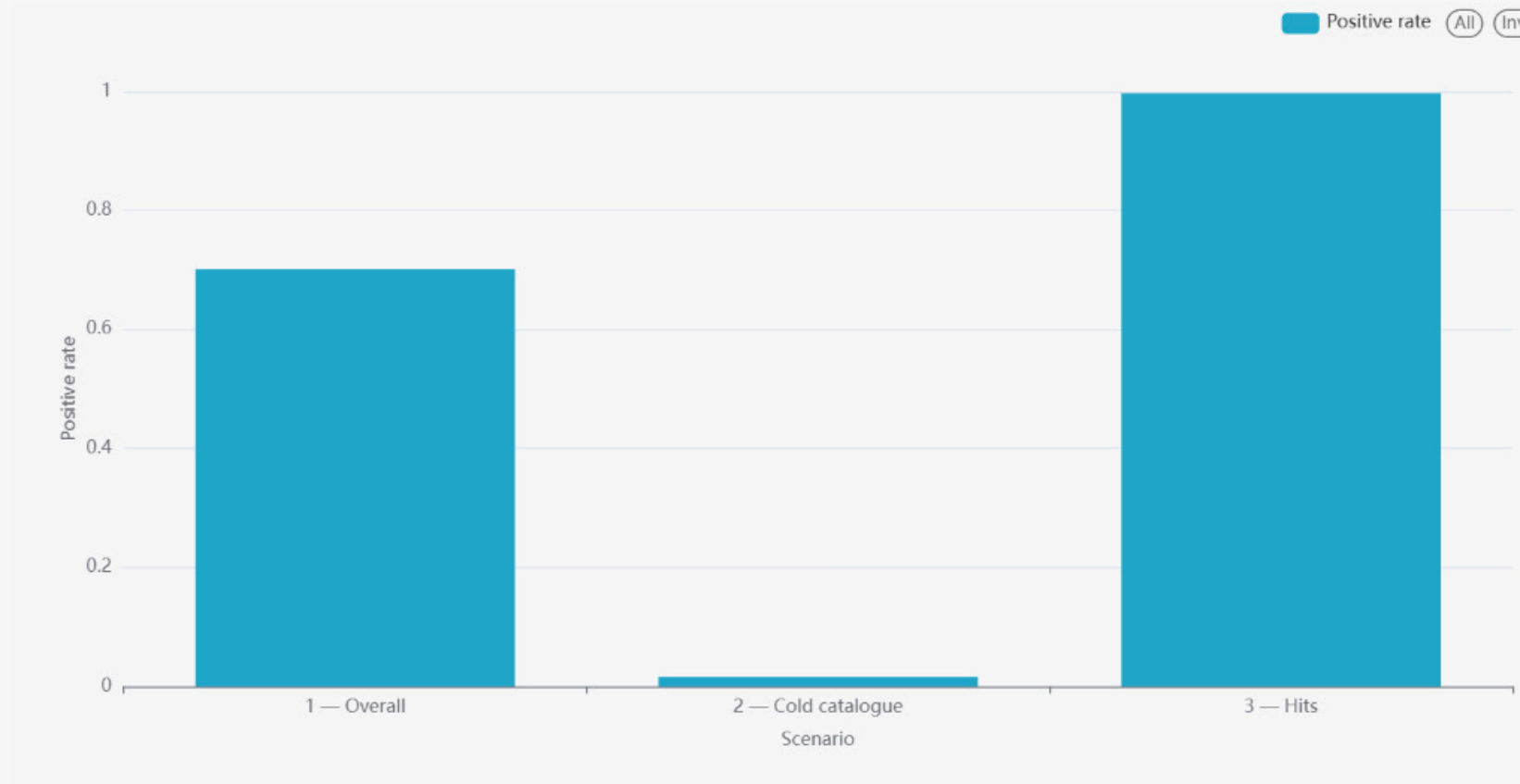
Stage II

Q5 – User activity distribution



Stage II

Q6 – Overall vs Cold
vs Hit positive rate



Stage III

User-disjoint split

Split strategy: $\text{hash}(\text{user_id}) \% 100$

Train: hash mod in [0,69] (70%)

Validation: [70,84] (15%)

Test: [85,99] (15%)

Users with < 2 interactions removed.

Inside each user, first 70% of interactions are **context**, remainder are **target rows**.

Sampled unobserved candidates (up to 30 per test user) added to test for ranking evaluation.

Stage III

Feature engineering pipeline

Group	Source	Key Transformers
Track audio	13 numeric columns	Imputer, VectorAssembler
Track metadata	explicit, key, mode, time_signature	StringIndexer → OneHotEncoder
Artist text	artists column	RegexTokenizer(pattern="\\W+") → HashingTF
Timestamp cyclic	ts (Unix bigint)	Decompose → sin/cos for month, day-of-week, hour
User history	last 50 context interactions	Aggregates: prior_user_interactions, positives, positive_share
User-artist history	same context window	Same three aggregates grouped by artist

Stage III

Evaluation results

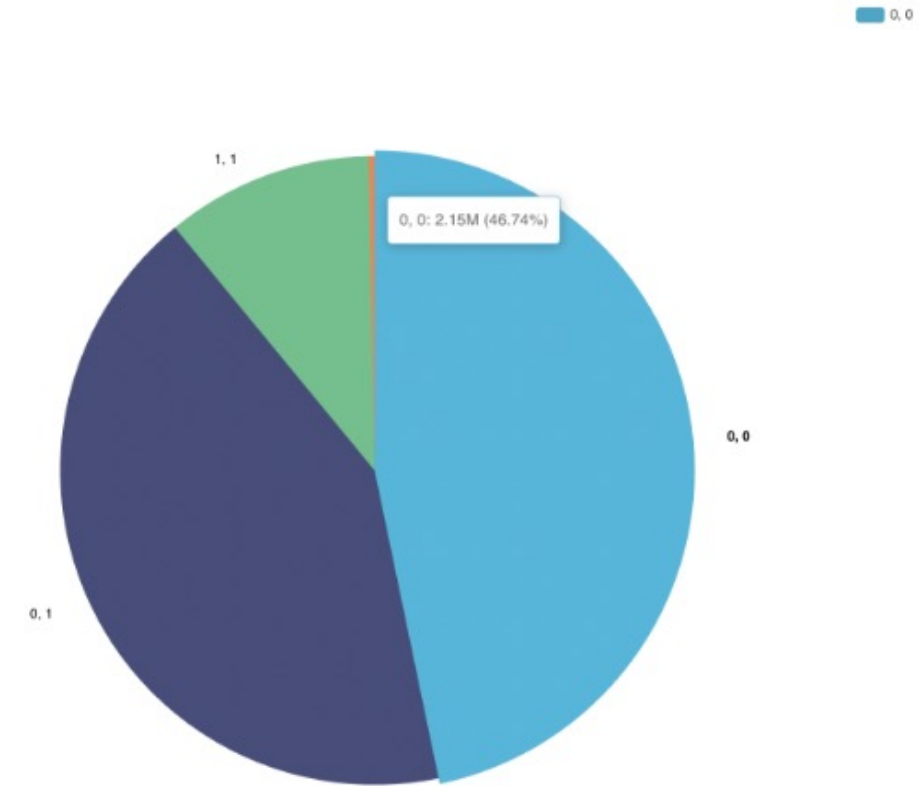
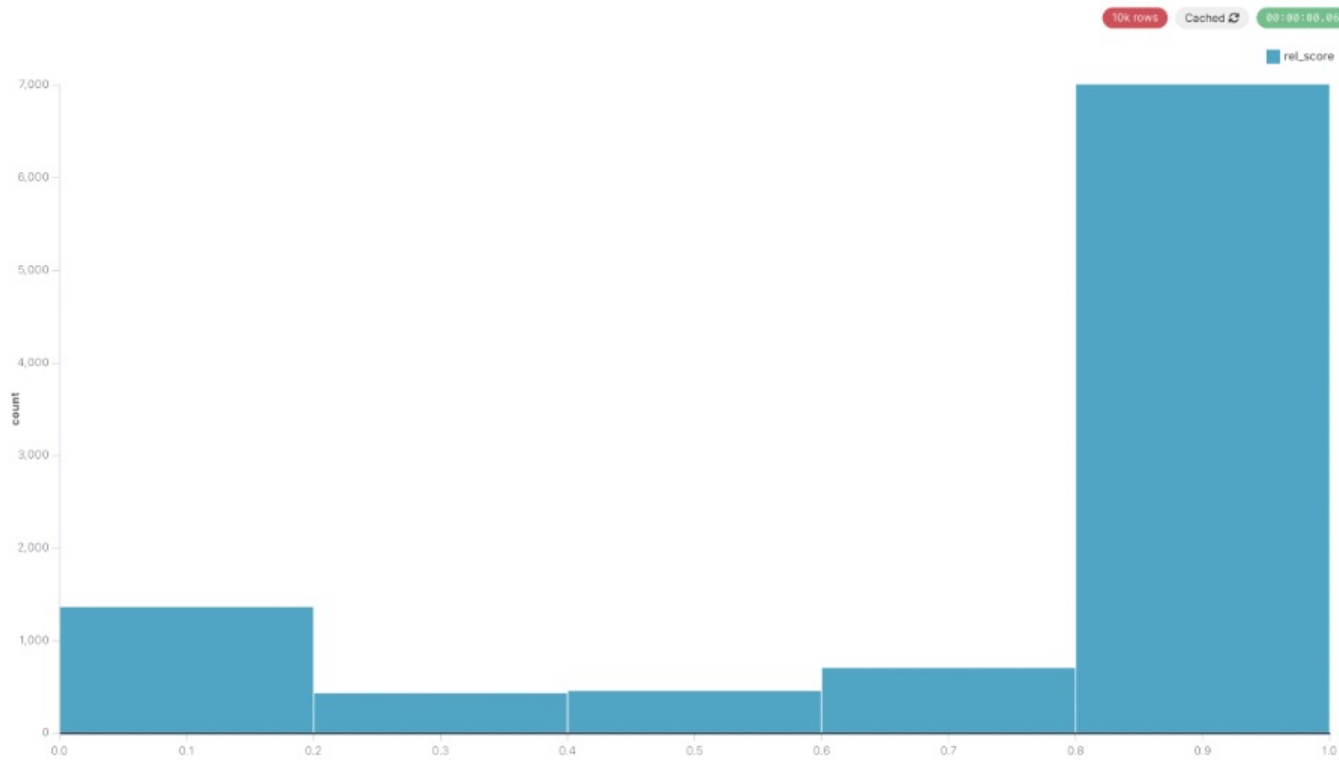
Test set: 133k users, 4.59M target rows + 4.01M sampled candidates

Baseline (always positive): Acc=0.11, Prec=0.11, F1=0.20

Metric	Logistic Regression	Random Forest
PR-AUC	0.4741	0.6285
ROC-AUC	0.8875	0.8725
Accuracy	0.5740	0.4861
Precision	0.2014	0.1731
Recall	0.9739	0.9770
F1	0.3337	0.2940
Precision@10	0.3565	0.3423
NDCG@10	0.7994	0.7693
MRR@10	0.8373	0.8137

Stage III

Logistic regression - score distribution & confusion matrix



Stage III

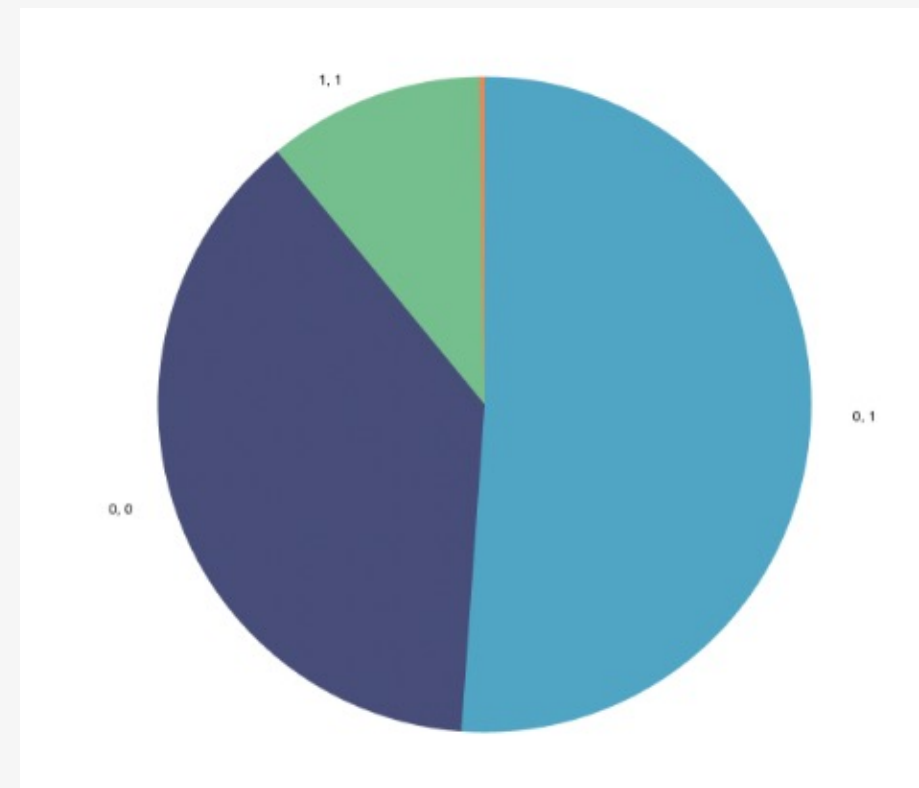
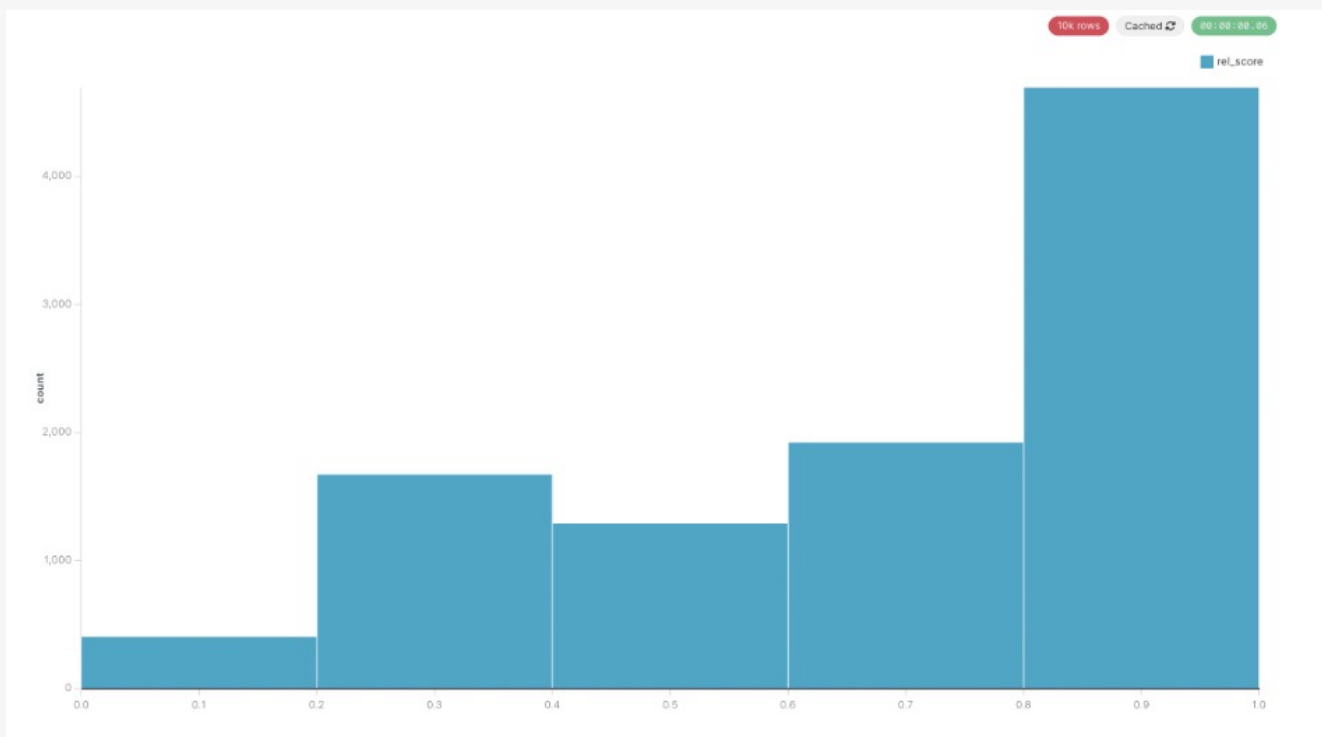
Logistic regression - score distribution & confusion matrix

Key observations:

1. Score distribution is strongly polarised and most predictions pushed near 0 or 1
2. Confusion matrix shows very low false negatives so recall > 97%
3. High false positives so precision ~0.20
4. Sharp scoring makes LogReg competitive on top-10 ranking metrics

Stage III

Random forest - score distribution & confusion matrix



Stage III

Random forest - score
distribution & confusion matrix

Key observations:

1. Score distribution is more spread through the middle range
2. Still skewed toward high scores, but less polarised than Logistic Regression
3. Confusion matrix: very low false negatives so recall > 97%
4. Larger false-positive share than Logistic Regression so precision ~0.17
5. Softer scoring improves overall PR-AUC but weakens strict-threshold precision

Dashboard

Challenges and solutions

Challenge

Pipeline reproducibility

User-disjoint split + ranking eval

Timestamp encoding

Imbalanced evaluation

Multi-system orchestration

Dashboard data freshness

Recommendation rather than classification

Unsuccessful ALS experiment

Our Solution

Scripts drop objects before creation, clear HDFS dirs, use IF NOT EXISTS

Custom hash-based user split; sampled unobserved candidates added to test only

Decompose into components, apply cyclic sin/cos transformation

Optimize **PR-AUC**; supplement with ranking metrics (NDCG, MRR)

Strict HDFS path conventions, Hive metastore as central catalog

Stage IV scripted external tables over Spark outputs

Built user history features from context window to personalize scores

We invested significant time trying Alternating Least Squares for collaborative filtering, but it did not yield competitive performance on our sparse user-artist matrix and was dropped in favor of supervised models. This taught us the importance of matching model to data density.

Conclusions

Conclusions

- 1. End-to-end pipeline delivered**
from CSV to dashboard, all on Hadoop cluster.
- 2. EDA directly informed ML design**
class imbalance, popularity bias, audio features all shaped the predictive stage.
- 3. Two distributed Spark models trained on YARN**
Random Forest best by PR-AUC (0.6285), Logistic Regression competitive on top-10 ranking.
- 4. Recommendation-style scoring maintained**
outputs `rel_score`, evaluated with ranking metrics on sampled candidate sets, user-history features make it personalized.
- 5. Reproducible, scripted, documented**
every stage rerunnable, code under version control, report and contribution table complete.

Predicting user preferences in a music service

